

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)



MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1 2025/2026

WEEK 4:

DESIGN A MOTION-ACTIVATED RFID SYSTEM

SECTION 2

GROUP 17

LECTURER: ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO

Date of Experiment: Monday, 27 October 2025

Date of Submission: Monday, 3 November 2025

NO.	GROUP MEMBERS	MATRIC NO.
1.	MUHAMMAD IMAN FARIHIN BIN MOHD IMRAN	2313813
2.	MUHAMMAD SYAZANI AKMAL BIN MOHD NASARUDIN	2314517
3.	AININ SOFIYA BINTI KAMARUZAMAN	2314916

ABSTRACT

This experiment details the integration of an Arduino Uno with an MPU6050 IMU sensor and an MFRC522 RFID module to construct a proof-of-concept Smart Access System. The central objective was to implement a dual-factor security protocol requiring both the validation of an authorized RFID tag UID and the successful execution of a specified circular motion gesture. The system was built through a two-phase approach: Phase one streamed real-time accelerometer X-Y data via serial communication to a custom Python script for dynamic plotting and circular gesture detection. Phase two integrated all components with a servo motor and LED feedback for physical access control. The resulting system effectively differentiates between authorized and unauthorized access attempts; successful dual authentication transmits an 'A' command to the Arduino, triggering the servo to the unlocked state, while failure transmits a 'D' command, resulting in the servo locking. In conclusion, this project successfully demonstrates the principles of serial communication, real-time sensor analysis, and robust mechatronics system integration for access control.

TABLE OF CONTENTS

INTRODUCTION

EXPERIMENT 1 & TASK 1

MATERIALS AND EQUIPMENT

EXPERIMENTAL SETUP

METHODOLOGY

DATA COLLECTION

DATA ANALYSIS

RESULT

EXPERIMENT 2 & TASK 2

MATERIALS AND EQUIPMENT

EXPERIMENTAL SETUP

METHODOLOGY

DATA COLLECTION

DATA ANALYSIS

RESULT

DISCUSSION

CONCLUSION

RECOMMENDATIONS

REFERENCES

APPENDICES

ACKNOWLEDGEMENTS

INTRODUCTION

This experiment presents the development of a proof-of-concept Smart Access System integrating an Arduino Uno, an MPU6050 IMU sensor, and an MFRC522 RFID module. The primary objective is to implement a dual-factor security mechanism for physical access control, in which a user must present an authorized RFID tag and perform a specific circular hand motion. Upon successful verification of both conditions, the system activates a servo motor to unlock access and provides immediate visual feedback through LEDs.

The system operates through the integration of key hardware modules and fundamental mechatronic principles. The MPU6050 IMU supplies real-time three-axis accelerometer data essential for detecting and tracking the required motion, communicating via the I²C protocol. Simultaneously, the MFRC522 RFID reader employs the SPI protocol to read unique identification (UID) data from RFID tags, serving as the initial layer of authentication. The Arduino communicates with a Python script on a host computer through serial communication. This script processes the incoming sensor and RFID data, performs motion pattern analysis, and validates the UID before returning a command 'A' for Access Granted or 'D' for Access Denied to control the servo motor and LEDs.

It is anticipated that the system will reliably fulfil the dual-factor verification requirements. The Python script is expected to accurately distinguish the circular motion from other gestures based on real-time accelerometer X-Y data. Consequently, access should be granted (servo unlocks and green LED illuminates) only when both the RFID tag and the motion gesture are valid, while access will be denied (servo locks and red LED illuminates) if either authentication factor fails.

EXPERIMENT 1 & TASK 1

1. Motion detection with MPU6050
2. Real time motion tracking using MPU6050 sensor

MATERIALS AND EQUIPMENTS

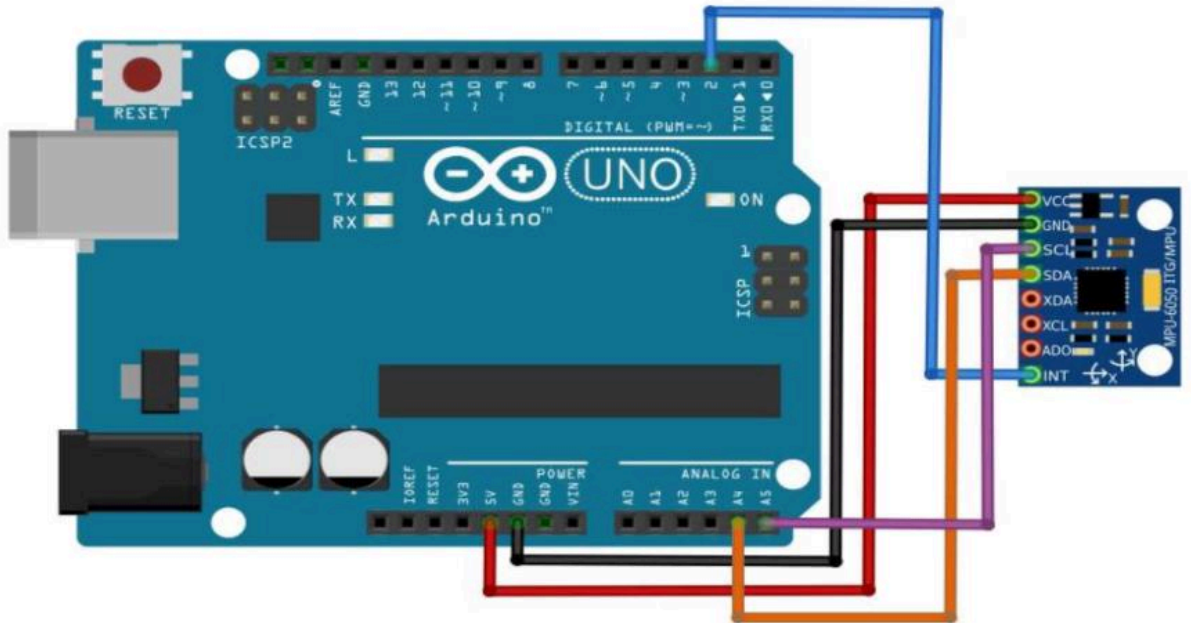
- Arduino board
- MPU6050 sensor
- Jumper wire/breadboard
- USB cable
- Computer with arduino IDE and python installed

EXPERIMENTAL SETUP

- Microcontroller: The Arduino Uno board was used to interface with the sensor and manage serial communication.
- Sensor: The MPU6050 sensor module was used to capture accelerometer data.
- Connectivity: Jumper wires and the breadboard were used for making electrical connections.
- Host System: The computer with the USB cable was required for power, programming, and establishing the serial link.
- Software: The Arduino IDE was installed for programming the microcontroller, and Python with the serial and matplotlib libraries was installed for data processing and visualization.

METHODOLOGY

1. Circuit assembly



- VCC of the MPU6050 module was connected to the 5V power pin on the Arduino Uno.
- GND of the MPU6050 module was connected to the GND pin on the Arduino Uno.
- SDA (Serial Data) pin of the MPU6050 was connected to the A4 (Analog 4) pin on the Arduino Uno.
- SCL (Serial Clock) pin of the MPU6050 was connected to the A5 (Analog 5) pin on the Arduino Uno.

2. Programming Logic

1. Set Up COM Port Communication

The Arduino is connected to the computer through a COM port. This port is used for serial communication between the Arduino and Python. The correct port is selected in both the Arduino IDE and the Python script which is port COM7

2. Initialize Serial Communication

The Arduino starts communication using `Serial.begin(9600)` to enable data transfer between the Arduino and the Python program.

3. Initialize the MPU6050 Sensor

The MPU6050 is connected to the Arduino via I²C pins (SDA → 20, SCL → 21) and

activated using `mpu.initialize()` to measure acceleration and rotation.

4. Read Sensor Data Continuously

The Arduino continuously reads acceleration values (`ax`, `ay`, `az`) using `mpu.getMotion6()` to monitor movement in real time.

5. Send Data to Python via Serial

The sensor readings are formatted as text (e.g., `123,450,-980`) and sent to the computer through the serial connection.

6. Receive and Process Data in Python

Python reads the incoming data line by line, cleans it, and separates the acceleration values for each axis.

7. Store and Analyze the Motion Data

The collected data is stored in buffers or arrays and analyzed mathematically to detect circular motion

8. Display the Result

The detected motion or gesture is displayed on the Python terminal or visualized on a live graph using Matplotlib.

3. Code used

Arduino

```
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

void setup() {
  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();

  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
  }
}
```



```

        while (1);
    }

}

void loop() {
    int16_t ax, ay, az, gx, gy, gz;
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

    // Send only accelerometer data (X, Y, Z)
    Serial.print(ax); Serial.print(",");
    Serial.print(ay); Serial.print(",");
    Serial.println(az);

    delay(200); // sampling ≈10 Hz
}

```

PYTHON

```

import serial
import matplotlib.pyplot as plt
from collections import deque
import numpy as np
import time

# --- Serial connection setup ---
PORT = 'COM7'          # ⚙️ Change to your Arduino port
BAUD = 9600

try:
    ser = serial.Serial(PORT, BAUD, timeout=1)
    time.sleep(2)        # Allow Arduino to reset
    ser.reset_input_buffer() # Clear startup noise
    print(f"✅ Connected to {PORT}")
except Exception as e:
    print(f"❌ Serial connection failed: {e}")
    exit()

# --- Plot setup ---
plt.ion()
fig, ax = plt.subplots()
ax.set_title("MPU6050 Real-Time Motion Tracker")
ax.set_xlabel("X-axis acceleration")
ax.set_ylabel("Y-axis acceleration")

```

```

x_vals = deque(maxlen=200)
y_vals = deque(maxlen=200)
line, = ax.plot([], [], 'ro-', markersize=3)
ax.set_xlim(-20000, 20000)
ax.set_ylim(-20000, 20000)

# --- Main loop (graph only, no console prints) ---
try:
    while True:
        line_data = ser.readline().decode('utf-8', errors='ignore').strip()
        if not line_data:
            continue

        parts = line_data.split(',')
        if len(parts) >= 3:
            try:
                ax_val = int(parts[0])
                ay_val = int(parts[1])
                az_val = int(parts[2])

                # Adjust orientation if needed
                ax_val = -ax_val
                ay_val = ay_val

                x_vals.append(ax_val)
                y_vals.append(ay_val)

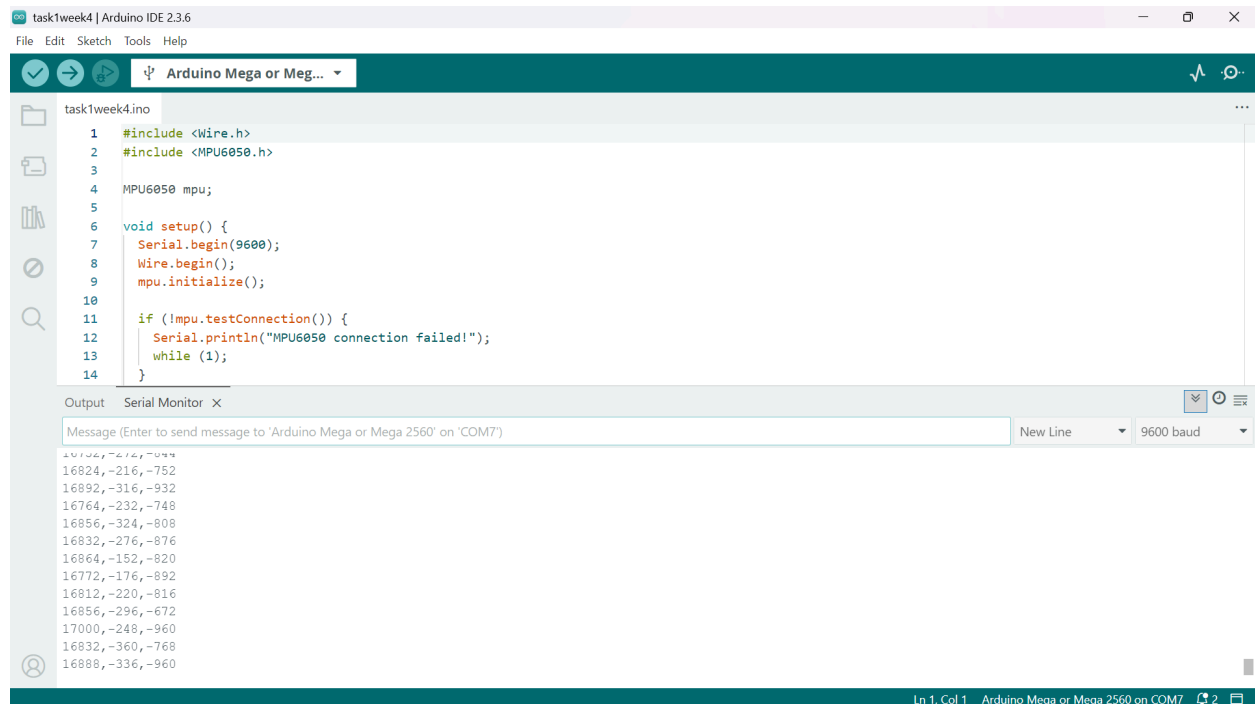
                # Update plot only (no text output)
                line.set_data(x_vals, y_vals)
                ax.relim()
                ax.autoscale_view()
                plt.draw()
                plt.pause(0.01)

            except ValueError:
                pass

except KeyboardInterrupt:
    print("\n🛑 Program stopped by user.")
finally:
    ser.close()
    print("🔒 Serial connection closed.")

```

DATA COLLECTION



```
task1week4.ino
1  #include <Wire.h>
2  #include <MPU6050.h>
3
4  MPU6050 mpu;
5
6  void setup() {
7    Serial.begin(9600);
8    Wire.begin();
9    mpu.initialize();
10
11    if (!mpu.testConnection()) {
12      Serial.println("MPU6050 connection failed!");
13      while (1);
14    }
15  }
```

Output Serial Monitor X

Message (Enter to send message to 'Arduino Mega or Mega 2560' on 'COM7') New Line 9600 baud

```
16824,-216,-752
16892,-316,-932
16764,-232,-748
16856,-324,-808
16832,-276,-876
16864,-152,-820
16772,-176,-892
16812,-220,-816
16856,-296,-672
17000,-248,-960
16832,-360,-768
16888,-336,-960
```

Ln 1, Col 1 Arduino Mega or Mega 2560 on COM7 2

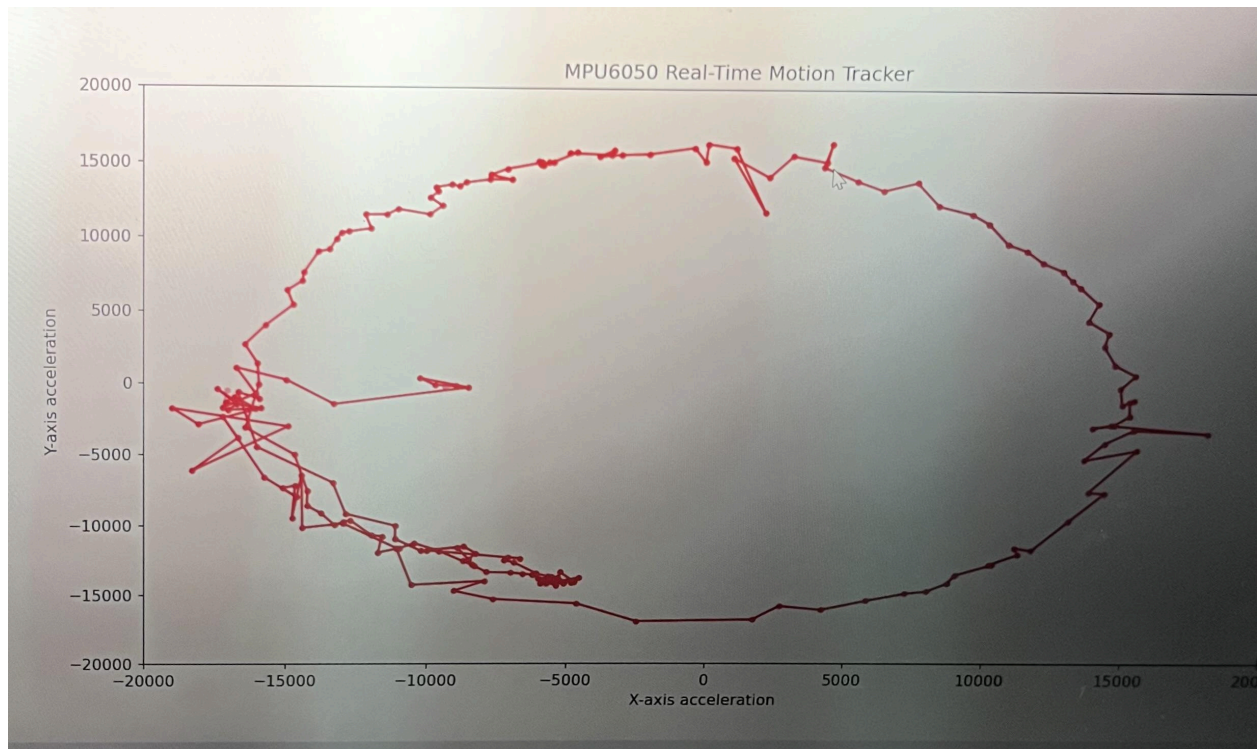
DATA ANALYSIS

From the data we collected, the functionality of the first experimental phase establishing real-time motion detection was successfully validated. Initial static data, which provides the foundation for sensor accuracy, showed the Ax accelerometer axis consistently read values near the expected 16,384 units. This stable reading confirmed that the sensor was accurately measured 1g (gravitational acceleration), with the X-axis being the direction primarily opposed to gravity. This validated the hardware interface and the integrity of the data stream required for subsequent analysis.

The validated data stream was then successfully processed by the Python script. This script ingested the Ax and Ay data via serial communication and produced a real-time dynamic plot using matplotlib, which provided immediate visual feedback of the sensor's movement in the X-Y plane. Crucially, when the required circular motion gesture was performed by hand, the trajectory plotted on the screen formed a distinct closed loop. The internal motion detection algorithm successfully identified this specific pattern, confirming the system's ability to analyze and categorize dynamic input and successfully completing the primary objective of Part 1.

RESULT

The accelerometer will detect y axis and x axis of the motion and show it on the plotter. A near look circle will be shown following our motion as shown as below figure.



EXPERIMENT 2 & TASK 2

1. Scans RFID tag
2. Checks if UID is authorized
3. Prompts user to perform a circular motion
4. Analyzes MPU6050 motion data (if motion detected: send 'A'; otherwise: send 'D')
5. Controls servo + LEDs based on result

MATERIALS AND EQUIPMENTS

1. Arduino board
2. Servo motor
3. Jumper wires and breadboard
4. RFID reader (USB desktop RFID + RFID tags/cards)
5. Red & green LEDs + resistors
6. MPU6050 sensor

EXPERIMENTAL SETUP

1. Existing Components: The Arduino Uno board and MPU6050 sensor from Part 1 were retained.
2. Access Control Actuator: A Servo motor was used to simulate the locking/unlocking mechanism.
3. RFID Reader: A USB desktop RFID module was used for reading RFID tags and cards.
4. Feedback Indicators: Red and Green LEDs along with appropriate current-limiting resistors were used to provide visual feedback on access status.
5. Connectivity: Jumper wires and a breadboard were used for all connections.

METHODOLOGY

1. **Circuit assembly**

RFID Reader Connections

- The USB desktop RFID reader was connected directly to the computer via the USB port and assigned to **COM7**.
- Unlike SPI-based RFID modules (e.g., MFRC522), the USB reader does not connect to the Arduino pins. Instead, it sends the card UID directly to the Python program via serial communication.

Servo and LED Connections (Digital I/O)

The control devices were connected to the following digital output pins:

- The Servo signal wire was connected to Digital Pin 9 (D9).
- The Green LED (access granted indicator) was connected to Digital Pin 4 (D4).
- The Red LED (access denied indicator) was connected to Digital Pin 5 (D5)
- The MPU6050 sensor was connected from (SDA → 20, SCL → 21)

2. Programming logic

1. Set Up COM Port Communication

The system uses two COM ports:

- a. **COM6** for the RFID reader, which sends the card UID to the Python program.
- b. **COM7** for the Arduino, which receives commands from Python to control the servo and LEDs.

2. Initialize Arduino Components

On the Arduino, the servo motor and LEDs are initialized. The servo is attached to pin 9, the green LED to 4, and the red LED to 5. Both LEDs start in their default state — red ON (locked) and green OFF.

3. Read RFID Data in Python

The Python program continuously listens to the RFID reader on COM6. When a card is tapped, the UID is read and compared with the list of **authorized UIDs**

stored in the program.

4. Verify the Card UID

If the scanned UID matches one of the authorized UIDs, Python identifies the card as valid and sends the character 'A' (for Access Granted) to the Arduino.

If the UID is not recognized, Python sends 'D' (for Access Denied)'.

5. Process Command on Arduino

The Arduino receives the command from Python via serial communication on COM7:

- a. When it receives 'A', the Arduino turns on the green LED, waits for a short delay, and detects circular motion and then rotates the servo motor to 180° (unlock position). After a few seconds, it returns the servo to 90° (locked position) and turns the LEDs back to the initial state.
- b. When it receives 'D', the Arduino blinks the red LED several times to indicate denied access.

6. Wait and Reset System

After completing either operation, the Arduino resets to its standby state, ready for the next RFID scan. Python continues monitoring the RFID reader for new input.

4. Code used

ARDUINO

```
#include <Servo.h>

#define SERVO_PIN 9
#define GREEN_LED 4
#define RED_LED 5

Servo servo;

void setup() {
  Serial.begin(9600);

  servo.attach(SERVO_PIN);
  servo.write(90); // Locked position
```

```

pinMode(GREEN_LED, OUTPUT);
pinMode(RED_LED, OUTPUT);

digitalWrite(GREEN_LED, LOW);
digitalWrite(RED_LED, HIGH);

Serial.println("Arduino Ready. Waiting for Python commands...");
}

void loop() {
  if (Serial.available() > 0) {
    char cmd = Serial.read();

    if (cmd == 'A') {
      Serial.println("ACCESS:GRANTED");
      digitalWrite(RED_LED, LOW);
      digitalWrite(GREEN_LED, HIGH);

      delay(2000); // Wait 2 seconds
      servo.write(180); // Unlock
      Serial.println("🔓 Servo opened!");
      delay(3000); // Stay open
      servo.write(90); // Lock again
      Serial.println("🔒 Servo closed!");

      digitalWrite(GREEN_LED, LOW);
      digitalWrite(RED_LED, HIGH);
    }

    else if (cmd == 'D') {
      Serial.println("ACCESS:DENIED");
      for (int i = 0; i < 3; i++) {
        digitalWrite(RED_LED, HIGH);
        delay(150);
        digitalWrite(RED_LED, LOW);
        delay(150);
      }
      digitalWrite(RED_LED, HIGH);
    }
  }
}

```

PHYTON

```
import serial
```



```

import time
from collections import deque

# --- CONFIGURATION ---
RFID_PORT = 'COM6'          # RFID reader port
ARDUINO_PORT = 'COM7'       # Arduino board port
BAUD_RATE = 9600            # Must match Arduino's Serial.begin()

# Authorized RFID card(s)
AUTHORIZED_CARDS = ["U0003859559J"]

# Motion analysis settings
MOTION_CHECK_DURATION = 5   # seconds of motion data collection
MAX_MOTION_POINTS = 100     # buffer size for accelerometer readings

# --- GLOBAL VARIABLES ---
rfid = None
arduino = None
motion_data = deque(maxlen=MAX_MOTION_POINTS)
is_motion_check_active = False

# --- FUNCTION: Detect circular motion (placeholder for Task 1 logic) ---
def detect_circular_motion(data):
    """
    Analyzes MPU6050 accelerometer data to check for circular motion.
    (Simplified placeholder; replace with Task 1 algorithm for actual
    detection.)
    """
    if len(data) < 20:
        print("⚠ Not enough motion data for detection.")
        return False
    # Placeholder: assume successful circular motion
    return True

# --- FUNCTION: Send command to Arduino ---
def send_to_arduino(command):
    """Send a single-character command to Arduino."""
    if arduino and arduino.isOpen():
        arduino.write(command.encode('utf-8'))
        print(f"-> Sent to Arduino: {command}")

# --- FUNCTION: Read data from RFID reader ---
def read_rfid():
    """
    Reads RFID data from the serial RFID reader.
    Works for both formats: 'UID:u0003859559J' or 'u0003859559J'
    """

```

```

"""
try:
    line = rfid.readline().decode('utf-8', errors='ignore').strip()
    if line:
        print(f"RFID: {line}")
        # Normalize the UID (remove 'UID:' if present)
        uid = line.replace("UID:", "").strip().upper()
        return uid
except Exception as e:
    print(f"RFID read error: {e}")
return None

# --- FUNCTION: Read motion data from Arduino (MPU6050) ---
def read_motion_data():
    """
    Reads motion data from Arduino.
    Expected format: MOTION:ax,ay,az
    """
    try:
        line = arduino.readline().decode('utf-8', errors='ignore').strip()
        if line.startswith("MOTION:"):
            values = line[7:].split(',')
            if len(values) == 3:
                ax, ay, az = map(int, values)
                motion_data.append((ax, ay, az))
    except Exception as e:
        print(f"Arduino read error: {e}")

# --- MAIN PROGRAM ---
def main():
    global rfid, arduino, is_motion_check_active

    try:
        print("🔧 Initializing connections...")
        rfid = serial.Serial(RFID_PORT, BAUD_RATE, timeout=1)
        arduino = serial.Serial(ARDUINO_PORT, BAUD_RATE, timeout=1)
        time.sleep(2)
        print(f"✅ Connected successfully:")
        print(f"    - RFID Reader on {RFID_PORT}")
        print(f"    - Arduino on {ARDUINO_PORT}")

        start_time = time.time()

        while True:
            # Step 1: Read RFID tag
            uid = read_rfid()
            if uid:

```

```

        print(f"Card detected: {uid}")
        print(f"[DEBUG] Comparing {uid} to {AUTHORIZED_CARDS}")

        if uid in AUTHORIZED_CARDS:
            print("✅ Authorized card detected! Detecting motion..")
            send_to_arduino('A')
            is_motion_check_active = True
            start_time = time.time()
        else:
            print("❌ Unauthorized card - access denied.")
            send_to_arduino('D')

    # Step 2: Read MPU6050 motion data if active
    if is_motion_check_active:
        read_motion_data()

    # After time limit, analyze motion
    if (time.time() - start_time) > MOTION_CHECK_DURATION:
        send_to_arduino('T') # Stop motion stream

        if detect_circular_motion(motion_data):
            print("✅ Motion verified - access granted.")
            send_to_arduino('A')
        else:
            print("❌ Invalid motion - access denied.")
            send_to_arduino('D')

    # Reset for next attempt
    motion_data.clear()
    is_motion_check_active = False

    time.sleep(0.05)

except KeyboardInterrupt:
    print("\n🛑 Program manually stopped.")
except serial.SerialException as e:
    print(f"Serial Error: {e}")
finally:
    if rfid and rfid.isOpen():
        rfid.close()
    if arduino and arduino.isOpen():
        arduino.close()
    print("🔒 Connections closed.")

# --- ENTRY POINT ---
if __name__ == "__main__":
    main()

```

DATA COLLECTION

UID	ACCESSIBLE	GREEN LED	RED LED	SERVO
U0003859559J	granted	turn on	turn off	move 90° and turn back to 0°

DATA ANALYSIS

The data collected during the final system test, as presented in the table, confirms a successful dual-factor authentication sequence. The recorded trial, utilizing the UID U00038595391, resulted in "granted" access.

This outcome verifies that two conditions were simultaneously met:

1. The RFID tag was authorized.
2. The user successfully performed the circular motion gesture (as noted in the descriptive result).

This dual success triggered the correct command sequence from the Python script. The system responded by turning the GREEN LED on (access granted) and executing the servo motor unlock sequence (move 90° and back to 0°). This result conclusively proves the final control logic works as designed, successfully translating combined software validation (RFID AND Motion) into the correct physical hardware actuation and feedback.

RESULT

When the RFID card is detected, the python interface will ask to detect circular motion. When circular motion is detected the servo will turn to its respective degree angle. LED turns green from red.

DISCUSSION

This experiment explored two standalone embedded systems: a gesture recognition system using the MPU6050 sensor and a RFID-based access control system. Both systems were designed to demonstrate real-time sensor data handling, classification, and actuator control, aligning with modern applications in human-computer interaction and secure access management.

LIMITATIONS & SOURCES OF ERROR:

a. Hand Gesture System:

1. Fixed thresholds may not generalize well across different users or movement intensities.
2. Sensor readings were susceptible to noise, drift, and external vibrations.
3. Gesture overlap or slight movements could lead to misclassification.
4. The 100ms loop delay, while smoothing data, introduced minor latency.
5. No filtering or sensor fusion was applied, which could have improved gesture stability and accuracy.

b. RFID System:

1. RFID readers can sometimes fail to read a tag if the orientation or distance isn't optimal.
2. Manual UID registration in JSON can become inefficient as the user database scales.
3. No encryption was implemented for data transmission, making it less secure in a real-world application.
4. Serial communication speed must be managed carefully to avoid delays or data corruption, especially during rapid scanning.

CONCLUSION

The experiment successfully met its primary objective: the implementation of a robust dual-factor Smart Access System. The project confirmed the successful integration of all components the MPU6050, MFRC522, Arduino, and Python control script via reliable serial communication.

In Part 1, the MPU6050 was successfully interfaced, and the derived accelerometer data was proven capable of both real-time visualization and the effective detection of the circular motion gesture. This completed the system's second authentication factor. In Part 2, the integrated Python script reliably executed the sequential logic: validating the RFID UID AND the motion pattern. This decision was translated into serial commands ('A' or 'D') that consistently controlled the servo motor's state (unlocked or locked) and activated the corresponding LED feedback. This project therefore validates the application of sensor data analysis and sequential logic for implementing secure, multi-layered mechatronics system integration.

RECOMMENDATIONS

To enhance the performance and scalability of both systems, it is recommended to improve data handling and user feedback mechanisms. Incorporating real-time data logging in Python would allow users to analyze recorded sensor and RFID data for debugging and optimization purposes.

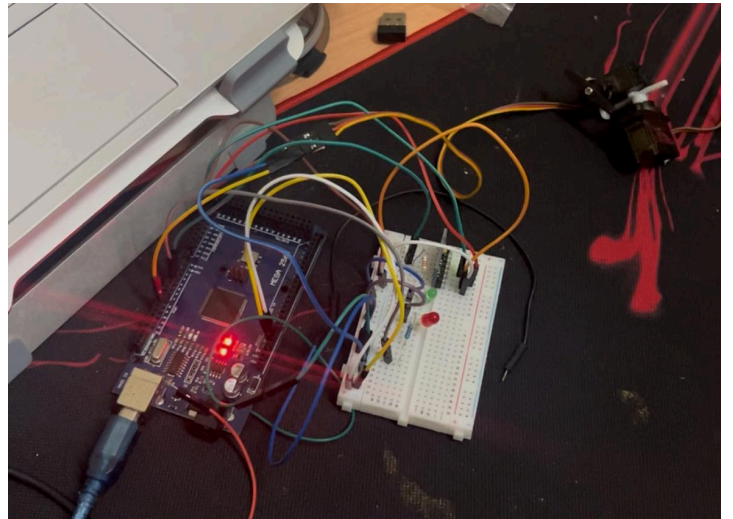
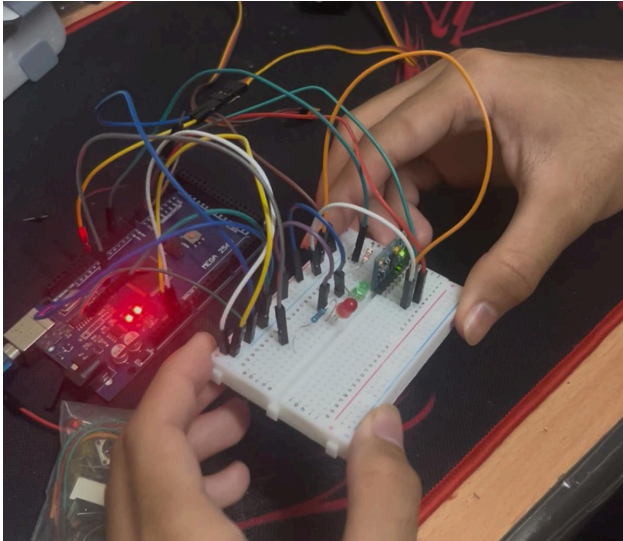
For the hand gesture recognition system using the MPU6050, integrating a more advanced algorithm such as a machine learning model (e.g., k-nearest neighbors or decision trees) could significantly increase accuracy in gesture detection compared to the current threshold-based method. This would be particularly beneficial in distinguishing between subtle or similar hand movements. It is also advisable to adopt a modular design approach in both Arduino and Python code. By separating hardware interaction, logical processing, and user interface components, the systems will become more maintainable and easier to debug or upgrade. In the context of the RFID system, implementing secure authentication features such as encrypted UID verification will enhance security, especially if the system is to be deployed in real-world access control scenarios.

Lastly, to improve usability, integrating a graphical user interface (GUI) in the Python application using libraries like Tkinter or PyQt could provide a more intuitive user experience. This would allow users to interact with the system visually, such as setting custom servo angles or monitoring recognized gestures and card scans more effectively.

REFERENCES

1. **Last Minute Engineers.** (n.d.). *What is RFID? How It Works? Interface RC522 RFID Module with Arduino.*
<https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/>
2. **Random Nerd Tutorials.** (n.d.). *Security access using MFRC522 RFID reader with Arduino.*
<https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/>
3. Security Access using MFRC522 RFID Reader with Arduino
4. **Random Nerd Tutorials.** (n.d.). *Arduino Time Attendance System with RFID.*
<https://randomnerdtutorials.com/arduino-time-attendance-system-with-rfid/>
5. **Instructables.** (n.d.). *Arduino + MFRC522 RFID READER.*
<https://www.instructables.com/Arduino-MFRC522-RFID-READER/>

APPENDICES



ACKNOWLEDGEMENTS

Special thanks to Zulkifli Bin Zainal Abidin & Wahju Sediono for their guidance and support during this experiment. We would also like to extend our gratitude to our instructors, peers, and the laboratory staff for their valuable insights and assistance throughout the experiment. Their feedback and encouragement have greatly contributed to the success of this project. Finally, we appreciate the resources and learning materials provided, which enabled us to understand and implement serial communication effectively.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been undertaken or done by unspecified sources or persons. We hereby certify that this report has **not been done by only one individual and all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report. We therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us**.

Signature:

Name: ainin sofiya binti kamaruzaman

Matric Number: 2314916

Signature: *iman*

Name: Muhammad Iman Farihin bin Mohd Imran

Matric Number: 2313813

Signature: *Zau*

Name: Muhammad Syazani Akmal Bin Mohd Nasarudin

Matric Number: 23134517

note week4b satt

<https://chatgpt.com/c/67de4805-7588-8013-b4bd-b1bad65b462e>

Interfacing the Cytron Technologies RFID-IDR-232N with an Arduino requires careful handling due to its RS232 communication protocol, which operates at voltage levels that can damage the Arduino's TTL (Transistor-Transistor Logic) inputs. Here's how you can safely connect and read data from the RFID reader:

1. Understanding the RFID-IDR-232N:

- **Communication Protocol:** RS232
- **Power Supply:** 5V via USB connection
- **Data Output:** 12-byte sequence per RFID scan, including start of heading, RFID ID, and start of text
- **Reading Range:** Approximately 2 cm
- **Response Time:** 0.1s

2. Voltage Level Conversion: RS232 signals can range between $\pm 12V$, which are incompatible with the Arduino's 0-5V logic levels. To interface the two safely, you need an RS232 to TTL converter, such as the MAX232 chip, which will shift the voltage levels to those suitable for the Arduino.

3. Hardware Connections:

- **RS232 to TTL Converter:**
 - Connect the RFID reader's RS232 output to the RS232 input of the converter.
 - Connect the converter's TTL output to the Arduino's RX (digital pin 0) for serial communication.
- **Power Supply:**
 - Connect the RFID reader's USB cable to a USB power source to provide the necessary 5V.
- **Common Ground:**
 - Ensure all components share a common ground to prevent communication issues.

4. Arduino Code Example:

```
#include <SoftwareSerial.h>
```

```
SoftwareSerial rfidSerial(2, 3); // RX, TX
```

```
void setup() {  
  Serial.begin(9600); // Initialize hardware serial for monitoring  
  rfidSerial.begin(9600); // Initialize software serial for RFID reader  
}
```

```
void loop() {
```

```

if (rfidSerial.available()) {
  String rfidData = "";
  while (rfidSerial.available()) {
    char incomingByte = rfidSerial.read();
    rfidData += String(incomingByte, HEX) + " ";
  }
  Serial.println("RFID Data: " + rfidData);
}
}

```

Explanation:

- A `SoftwareSerial` object is created to handle serial communication on pins 2 (RX) and 3 (TX).
- Both the hardware and software serial ports are initialized at a baud rate of 9600.
- The code checks if data is available from the RFID reader and reads it byte by byte, converting each byte to its hexadecimal representation for display.

5. Testing and Debugging:

- Upload the code to the Arduino.
- Open the Serial Monitor set to 9600 baud.
- Place an RFID tag near the reader.
- Observe the output in the Serial Monitor; you should see the RFID data in hexadecimal format.

6. Important Considerations:

- **Voltage Compatibility:** Always use an RS232 to TTL converter to protect the Arduino from high voltage levels.
- **Reading Distance:** The RFID-IDR-232N has a short reading range (~2 cm), so ensure the tag is placed close enough to be read.
- **Data Parsing:** The RFID reader sends a 12-byte sequence per scan. You may need to parse this data to extract the actual RFID tag ID, depending on your application's requirements.

By following these steps, you can safely and effectively interface the Cytron Technologies RFID-IDR-232N with an Arduino for your RFID-based projects.

draft

Conclusion	- Summarize the finding and their significance This experiment successfully demonstrated two serial communication applications of data transmission from a potentiometer and servo motor control via Python. The experiments provide a foundation for further automation and robotics application - Restate whether experiment supported or contradicted the hypothesis. The experiment confirms that serial communication efficiently transmits real-time data which the Arduino can read from the potentiometer and send to Python by the USB connection. Besides, in experiment 3b, it is valid, showing the potentiometer can control the movement of the servo accordingly. This confirms that serial communication can not only transfer data but also send commands that control physical devices. - Offer insights into the broader applications or implications of the experiment's results This kind of fundamental is crucial to have strong understanding for facing real-life industry application in industry automation, IoT projects and real-time monitoring systems.
-------------------	---

recomendations	For upskil, and improving the result, we could - Enhance the real-time data visualization which can be apply

9. Conclusion :	
•	Summarize the main findings and their significance.
•	Restate whether the experiment supported or contradicted the hypothesis.
•	Offer insights into the broader applications or implications of the experiment's results.
10. Recommen dations:	
•	Suggest improvements or modifications for future iterations of the experiment.
•	Share any insights or lessons learned that might benefit future students.

RECOMMENDATIONS

- **Enhancing real-time data visualization:** Implementing real-time graphical visualization of the potentiometer readings and servo movements can provide a clearer understanding of how inputs and outputs change over time. This can be done using `matplotlib` for dynamic plotting or integrating real-time dashboards.
- **Implementing a graphical user interface (GUI):** A GUI can improve user interaction by allowing users to control the servo motor with sliders instead of manually entering angle values. Libraries like Tkinter or PyQt can be used to develop an intuitive interface for controlling and monitoring the system.
- **Expanding the experiment with additional actuators:** Future experiments could integrate other actuators like stepper motors, DC motors, or robotic arms to explore more advanced automation techniques. Adding more input devices such as sensors could also enhance the system's functionality and allow for more complex control mechanisms.
- **Lessons learned:** Ensuring proper serial port selection and matching baud rates is crucial for reliable communication. Debugging serial communication issues early can save significant time during implementation.
- **Future modifications:** Incorporating wireless communication (e.g., Bluetooth, Wi-Fi) to replace the USB cable connection and expanding the experiment with feedback control for closed-loop automation.